qual_by_position.py 코드 해설

> lab11 · 생물정보학실험
> 대상 파일: `lab11/src/qual_by_position.py`
> 실행 방법과 결과 해석은 `qual_by_position_usage.md`를 참고하세요.

---

1. 코드 전문

```
import sys

def read_fastq(filename):
    records = []
    with open(filename) as f:
        lines = f.readlines()

    for i in range(0, len(lines), 4):
        name = lines[i].rstrip()[1:]
        seq = lines[i + 1].rstrip()
        qual = lines[i + 3].rstrip()
        records.append((name, seq, qual))

    return records


reads = read_fastq(sys.argv[1])

# 각 위치의 품질 점수 합계와 read 개수
quality_sums = []
quality_counts = []

for name, seq, qual in reads:
    for i, char in enumerate(qual):

        # 새로운 위치가 나오면 저장 공간 추가
        if i >= len(quality_sums):
            quality_sums.append(0)
            quality_counts.append(0)

        # Phred+33 방식으로 품질 점수 계산
        score = ord(char) - 33

        quality_sums[i] += score
        quality_counts[i] += 1

# 위치별 평균 품질 점수 출력
print("위치\t평균 품질 점수\tRead 수")

for i in range(len(quality_sums)):
    mean = quality_sums[i] / quality_counts[i]
    print(f"{i + 1}\t{mean:.2f}\t{quality_counts[i]}")
```

코드는 크게 네 부분으로 나뉩니다.

부분	하는 일
① 준비	`sys` 모듈 불러오기
② `read_fastq()` 함수	FASTQ 파일을 읽어 (이름, 서열, 품질) 목록 만들기
③ 위치별 집계	위치마다 품질 점수 합계와 read 수 세기
④ 출력	위치마다 평균을 계산해 표로 출력

2. ① 준비

```
import sys
```

코드	해설
`import sys`	명령줄에서 입력한 값(파일 이름)을 읽기 위해 `sys` 모듈을 불러온다. 나중에 `sys.argv[1]`로 파일 이름을 꺼낸다.

3. ② `read_fastq()` 함수

```
def read_fastq(filename):
    records = []
    with open(filename) as f:
        lines = f.readlines()

    for i in range(0, len(lines), 4):
        name = lines[i].rstrip()[1:]
        seq = lines[i + 1].rstrip()
        qual = lines[i + 3].rstrip()
        records.append((name, seq, qual))

    return records
```

코드	해설
`def read_fastq(filename):`	파일 이름을 받아 FASTQ를 읽는 함수를 만든다.
`records = []`	결과를 담을 빈 리스트를 만든다.
`with open(filename) as f:`	파일을 연다. `with`를 쓰면 다 쓴 뒤 파일이 자동으로 닫힌다.
`lines = f.readlines()`	파일의 모든 줄을 한꺼번에 읽어 리스트로 만든다. `lines[0]`이 첫 줄.
`for i in range(0, len(lines), 4):`	i를 0, 4, 8, …로 4씩 늘린다. FASTQ는 **4줄이 read 하나**이므로 i는 항상 각 read의 첫 줄 번호다.
`name = lines[i].rstrip()[1:]`	1번째 줄(`@`이름). `rstrip()`으로 줄바꿈을 지우고 `[1:]`로 맨 앞 `@`를 뗀다.
`seq = lines[i + 1].rstrip()`	2번째 줄: 염기 서열.
`qual = lines[i + 3].rstrip()`	4번째 줄: 품질 문자열. 3번째 줄(`+`)은 필요 없어서 건너뛴다.
`records.append((name, seq, qual))`	세 값을 튜플 하나로 묶어 리스트에 추가한다.
`return records`	완성된 리스트를 돌려준다.

예를 들어 실습 파일의 첫 read는 아래 튜플이 됩니다.

```
("SRR1553607.1 1 length=101",
 "GTTAGCGTTGTTGATCGC...",
 "BB@FFFFFHHHHHJJJJJ...")
```

---

4. ③ 위치별 집계

```
reads = read_fastq(sys.argv[1])

quality_sums = []
quality_counts = []
```

코드	해설
`reads = read_fastq(sys.argv[1])`	명령줄의 첫 번째 값(파일 이름)으로 함수를 불러 모든 read를 `reads`에 저장한다. 파일 이름을 안 주면 여기서 `IndexError`가 난다.
`quality_sums = []`	위치별 **품질 점수 합계**를 담을 리스트. `quality_sums[0]`이 1번 위치 합계.
`quality_counts = []`	위치별 **read 개수**를 담을 리스트.

처음에는 두 리스트 모두 비어 있습니다. read 길이를 미리 모르기 때문에, 읽으면서 필요한 만큼 늘려 갑니다.

```
for name, seq, qual in reads:
    for i, char in enumerate(qual):
```

코드	해설
`for name, seq, qual in reads:`	read를 하나씩 꺼내고, 튜플의 세 값을 각각 `name`, `seq`, `qual`에 나눠 담는다. 이 코드에서 실제로 쓰는 것은 `qual`뿐이다.
`for i, char in enumerate(qual):`	품질 문자열을 한 글자씩 본다. `enumerate`는 **위치 번호 i**(0부터)와 **문자 char**를 함께 준다. 예: `"BB@"` → (0,`B`), (1,`B`), (2,`@`).

```
        if i >= len(quality_sums):
            quality_sums.append(0)
            quality_counts.append(0)
```

코드	해설
`if i >= len(quality_sums):`	지금 위치 i에 해당하는 칸이 아직 없는지 확인한다. 예: 리스트 길이가 3이면 0~2번 칸만 있으므로 i=3이면 칸이 없다.
`quality_sums.append(0)`	합계 칸을 0으로 하나 추가한다.
`quality_counts.append(0)`	개수 칸도 0으로 하나 추가한다.

이 덕분에 **가장 긴 read의 길이만큼** 칸이 자동으로 생깁니다. 짧은 read와 긴 read가 섞여 있어도 문제없습니다.

```
        score = ord(char) - 33

        quality_sums[i] += score
        quality_counts[i] += 1
```

코드	해설
`score = ord(char) - 33`	품질 문자를 숫자 점수로 바꾼다. `ord()`는 문자의 ASCII 번호를 주고, Phred+33 규칙에 따라 33을 뺀다. 예: `ord('J')` = 74 → 74 − 33 = **41점**.
`quality_sums[i] += score`	i번 위치의 합계에 이 점수를 더한다.
`quality_counts[i] += 1`	i번 위치의 read 수를 1 늘린다.

짧은 read는 자기 길이까지만 `quality_counts`를 올리므로, 위치마다 **그 위치를 실제로 가진 read 수**가 따로 기록됩니다.

---

5. ④ 출력

```
print("위치\t평균 품질 점수\tRead 수")

for i in range(len(quality_sums)):
    mean = quality_sums[i] / quality_counts[i]
    print(f"{i + 1}\t{mean:.2f}\t{quality_counts[i]}")
```

코드	해설
`print("위치\t평균 품질 점수\tRead 수")`	표의 제목 줄을 출력한다. `\t`는 탭(칸 나누기) 문자다.
`for i in range(len(quality_sums)):`	0번부터 마지막 위치까지 하나씩 돈다.
`mean = quality_sums[i] / quality_counts[i]`	합계 ÷ read 수 = 그 위치의 **평균 품질 점수**. 전체 read 수가 아니라 그 위치의 read 수로 나누므로 짧은 read 때문에 평균이 낮아지지 않는다.
`print(f"{i + 1}\t{mean:.2f}\t{quality_counts[i]}")`	위치 번호(사람은 1부터 세므로 `i + 1`), 평균(소수점 둘째 자리 `:.2f`), read 수를 탭으로 구분해 출력한다.

출력 예시 (`data/SRR1553607_1.fastq`):

```
위치  평균 품질 점수    Read 수
1   32.04   203445
2   32.17   203445
3   32.28   203445
...
101 28.25   203445
```

6. 한눈에 보는 흐름

```
FASTQ 파일
    │  read_fastq()  : 4줄씩 끊어 (이름, 서열, 품질) 목록 만들기
    ▼
reads 리스트
    │  read마다, 품질 문자마다
    │    - 칸이 없으면 추가
    │    - ord(문자) - 33 → 점수
    │    - quality_sums[i] += 점수, quality_counts[i] += 1
    ▼
위치별 합계 / 개수
    │  합계 ÷ 개수
    ▼
위치별 평균 품질 점수 표 출력
```

7. 알아 두면 좋은 점

- **`name`, `seq`는 쓰이지 않습니다.** 품질만 계산하는 프로그램이라서 그렇습니다. `read_fastq()`를 다른 프로그램(`qual_summary.py` 등)과 같은 모양으로 맞춰 두었기 때문에 함께 읽어 오는 것입니다.
- **파일 전체를 메모리에 올립니다.** `readlines()`가 모든 줄을 한 번에 읽으므로, 아주 큰 FASTQ 파일에서는 메모리가 부족할 수 있습니다.
- **FASTQ 형식이 정확해야 합니다.** 빈 줄이 끼어 있거나 줄 수가 4의 배수가 아니면 `IndexError`가 나거나 엉뚱한 줄을 품질로 읽게 됩니다.